

■ MEAN STACK

Node.js Backend Developer Guide

From JavaScript Basics to Full-Stack Web Apps

MongoDB · Express.js · Angular · Node.js

Table of Contents

1.	What is Node.js?	3
2.	Setting Up Your Environment	3
3.	Your First Node.js Server	4
4.	npm — Node Package Manager	4
5.	Express.js — Building REST APIs	5
6.	MongoDB — The Database	6
7.	Mongoose — Connecting Node to MongoDB	7
8.	Building a Full REST API (CRUD)	8
9.	Angular — The Frontend	9
10.	Connecting Angular to Express API	10
11.	MEAN Stack Project Structure	11
12.	Quick Reference Cheat Sheet	12

1 What is Node.js?

Node.js lets you run JavaScript on the server — outside the browser. Before Node.js, JavaScript only worked in browsers. Now, with Node.js, you can build web servers, APIs, command-line tools, and full-stack apps using the same language you already know: JavaScript.

Key concepts:

- **V8 Engine:** Node.js uses Google Chrome's V8 engine to run JavaScript at blazing speed.
- **Non-blocking I/O:** Node.js handles many requests at once without waiting — perfect for APIs.
- **Single-threaded:** One thread handles everything using an event loop, keeping memory usage low.
- **NPM Ecosystem:** Access 2 million+ packages to add features without coding from scratch.

■ Think of it this way

Browser JavaScript controls your webpage. Node.js JavaScript controls your server. Same language, different superpowers!

2 Setting Up Your Environment

Step 1 — Install Node.js

Go to nodejs.org and download the LTS (Long Term Support) version. This also installs npm automatically.

Step 2 — Verify installation

■ Open your terminal and run:

```
node --version
npm --version
```

Step 3 — Install VS Code

Download Visual Studio Code from code.visualstudio.com — it's the best editor for JavaScript and Node.js development.

Step 4 — Recommended VS Code Extensions

- ESLint
- Prettier
- REST Client
- MongoDB for VS Code
- Angular Language Service

3 Your First Node.js Server

Let's create a simple HTTP server without any libraries — just pure Node.js using the built-in `http` module.

■ server.js — your first Node server

```
// Import Node's built-in http module
const http = require('http');

// Create the server
```

```
const server = http.createServer((req, res) => {  
  res.writeHead(200, { 'Content-Type': 'text/plain' });  
  res.end('Hello from Node.js!');  
});  
  
// Listen on port 3000  
server.listen(3000, () => {  
  console.log('Server running at http://localhost:3000');  
});
```

■ Run it:

```
node server.js
```

Open your browser and go to <http://localhost:3000> — you'll see your message. Congratulations, you're a backend developer! ■

■ require() vs import

In older Node.js you use `require()`. In modern Node.js (ES Modules) you use `import`. Most Express tutorials still use `require()`, so we'll use that here.

4 npm — Node Package Manager

npm lets you install open-source packages (libraries) into your project. Think of it like an app store for JavaScript code.

■ Essential npm commands

```
npm init -y # Create a new project (generates package.json)
npm install express # Install a package
npm install # Install all packages from package.json
npm install -D nodemon # Install as dev-only dependency
npm uninstall express # Remove a package
npm run start # Run a script defined in package.json
```

What is package.json?

Every Node.js project has a `package.json` file. It records your project name, version, and all the packages it depends on.

■ package.json example

```
{
  "name": "my-mean-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^8.0.0"
  }
}
```

■ Use nodemon for development

nodemon auto-restarts your server whenever you save a file. Install it globally: `npm install -g nodemon`, then run: `nodemon server.js`

5 Express.js — Building REST APIs

Express.js is a minimal web framework for Node.js. It makes routing, handling requests, and sending responses much simpler than raw Node.js. It is the **E** in MEAN.

■ Install Express

```
npm init -y
npm install express
```

Basic Express Server

■ app.js

```
const express = require('express');
const app = express();

// Middleware: parse JSON request bodies
app.use(express.json());

// Routes
app.get('/', (req, res) => {
  res.send('Welcome to my API!');
});

app.get('/hello/:name', (req, res) => {
  res.json({ message: `Hello, ${req.params.name}!` });
});

app.listen(3000, () => console.log('API running on port 3000'));
```

HTTP Methods in Express

Method	Example Route	Purpose
GET	/users	Read / fetch data
POST	/users	Create new data
PUT	/users/:id	Update existing data
DELETE	/users/:id	Delete data

Middleware

Middleware functions run between the request arriving and your route handler running. They can check authentication, log requests, parse data, and more.

■ Custom middleware example

```
// Logger middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next(); // Pass control to the next middleware/route
});
```

6 MongoDB — The Database

MongoDB is a NoSQL database. Instead of tables and rows (like SQL), it stores data as **documents** (JSON-like objects) in **collections**. It is the **M** in MEAN.

SQL vs MongoDB

SQL Term	MongoDB Term	Example
Database	Database	myapp
Table	Collection	users
Row	Document	{ name: "Ali", age: 25 }
Column	Field	name, age, email
Primary Key	_id	"_id": ObjectId("abc123")

Setting Up MongoDB

- Option A: Install locally from mongodb.com/try/download
- Option B (Recommended): Use MongoDB Atlas — free cloud database at cloud.mongodb.com
- Create a free cluster, get your connection string, and you're ready to go.

■ A MongoDB document looks like this (plain JSON)

```
{
  "_id": "64abc123",
  "name": "Alice",
  "email": "alice@example.com",
  "age": 28,
  "hobbies": ["coding", "reading"],
  "address": {
    "city": "Dhaka",
    "country": "Bangladesh"
  }
}
```

■ Why MongoDB with Node.js?

MongoDB stores data as JSON, and Node.js speaks JSON natively. They're a perfect match — no awkward data conversion required!

7 Mongoose — Connecting Node to MongoDB

Mongoose is an ODM (Object Document Mapper). It lets you define schemas for your data and gives you easy methods to create, read, update, and delete documents.

■ Install Mongoose

```
npm install mongoose
```

■ db.js — Connect to MongoDB

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/myapp')
  .then(() => console.log('MongoDB connected!'))
  .catch(err => console.error('Connection error:', err));
```

Defining a Schema and Model

A schema defines the structure of your documents. A model is the class you use to interact with the collection.

■ models/User.js

```
const mongoose = require('mongoose');

// Define the schema
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  age: { type: Number, min: 0 },
  createdAt: { type: Date, default: Date.now }
});

// Create and export the model
module.exports = mongoose.model('User', userSchema);
```

■ Using the model — CRUD operations

```
const User = require('./models/User');

// CREATE
const newUser = new User({ name: 'Alice', email: 'alice@example.com', age: 25 });
await newUser.save();

// READ all
const users = await User.find();

// READ one
const user = await User.findById('64abc123');

// UPDATE
await User.findByIdAndUpdate('64abc123', { age: 26 });

// DELETE
await User.findByIdAndDelete('64abc123');
```


8 Building a Full REST API (CRUD)

Let's put it all together and build a complete Users API with Express + Mongoose. This is the backend of a real MEAN app.

■ routes/users.js — Complete CRUD Routes

```
const express = require('express');
const router = express.Router();
const User = require('../models/User');

// GET all users
router.get('/', async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// GET single user
router.get('/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ error: 'Not found' });
    res.json(user);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// POST create user
router.post('/', async (req, res) => {
  try {
    const user = new User(req.body);
    await user.save();
    res.status(201).json(user);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// PUT update user
router.put('/:id', async (req, res) => {
  try {
```

```
const user = await User.findByIdAndUpdate(
  req.params.id, req.body, { new: true }
);
res.json(user);
} catch (err) {
  res.status(400).json({ error: err.message });
}
});

// DELETE user
router.delete('/:id', async (req, res) => {
  try {
    await User.findByIdAndDelete(req.params.id);
    res.json({ message: 'User deleted' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
```

■ server.js — Wire everything together

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();

app.use(express.json());

// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/meanapp')
  .then(() => console.log('DB connected'));

// Mount routes
app.use('/api/users', require('./routes/users'));

app.listen(3000, () => console.log('Server on port 3000'));
```

■ Test your API with REST Client or Postman

Use Postman (postman.com) or the VS Code REST Client extension to send GET, POST, PUT, DELETE requests to your API and see the responses.

9 Angular — The Frontend

Angular is a powerful TypeScript-based frontend framework built by Google. It is the **A** in MEAN. Angular organises your UI into components, services, and modules.

■ Install Angular CLI and create a project

```
npm install -g @angular/cli
ng new my-mean-app
cd my-mean-app
ng serve
```

Core Angular Concepts

- **Component:** A piece of UI — has HTML template, CSS styles, and TypeScript logic.
- **Service:** Shared logic or data fetching — keeps components clean.
- **Module:** Groups related components and services together.
- **Routing:** Navigates between pages without a full page reload.
- **HttpClient:** Built-in service for making HTTP requests to your Express API.

■ A simple Angular component

```
// user-list.component.ts
import { Component, OnInit } from '@angular/core';
import { UserService } from '../user.service';

@Component({
  selector: 'app-user-list',
  template: `
    Users

    {{ user.name }}
  `
})
export class UserListComponent implements OnInit {
  users: any[] = [];

  constructor(private userService: UserService) {}

  ngOnInit() {
    this.userService.getUsers().subscribe(data => this.users = data);
  }
}
```

10 Connecting Angular to Express API

Angular's **HttpClient** service lets you call your Express API. First, enable it in your app module, then create a service.

■ app.module.ts — Enable HttpClient

```
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [ HttpClientModule, ... ]
})

export class AppModule {}
```

■ user.service.ts — Fetch users from Express

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class UserService {
  private apiUrl = 'http://localhost:3000/api/users';

  constructor(private http: HttpClient) {}

  getUsers() { return this.http.get(this.apiUrl); }
  getUserById(id: string) { return this.http.get(`${this.apiUrl}/${id}`); }
  createUser(user: any) { return this.http.post(this.apiUrl, user); }
  updateUser(id: string, user: any) {
    return this.http.put(`${this.apiUrl}/${id}`, user); }
  deleteUser(id: string) { return this.http.delete(`${this.apiUrl}/${id}`); }
}
```

■ CORS Error? Fix it in Express

Install cors: `npm install cors`, then add: `app.use(require("cors")())` in `server.js` This allows Angular (port 4200) to talk to Express (port 3000).

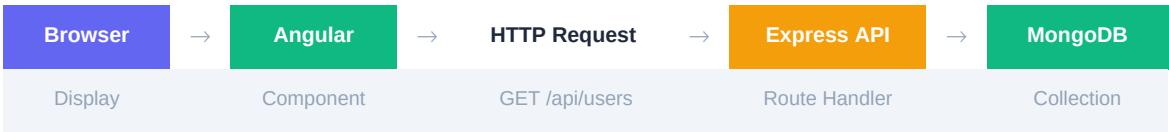
11 MEAN Stack Project Structure

A clean project structure makes your code easier to navigate and maintain. Here is a recommended layout for a full MEAN stack application:

■ Recommended folder structure

```
mean-app/  
  ■■■ backend/ ← Node.js + Express + MongoDB  
  ■ ■■■ models/  
  ■ ■ ■■■ User.js ← Mongoose schemas  
  ■ ■■■ routes/  
  ■ ■ ■■■ users.js ← Express route handlers  
  ■ ■■■ middleware/  
  ■ ■ ■■■ auth.js ← JWT auth middleware  
  ■ ■■■ db.js ← MongoDB connection  
  ■ ■■■ server.js ← Entry point  
  ■ ■■■ package.json  
  ■  
  ■■■ frontend/ ← Angular app  
  ■■■ src/  
  ■ ■■■ app/  
  ■ ■ ■■■ components/ ← UI components  
  ■ ■ ■■■ services/ ← HttpClient services  
  ■ ■ ■■■ app.module.ts  
  ■ ■■■ environments/  
  ■■■ package.json
```

How it all fits together



12 Quick Reference Cheat Sheet

Command / Concept	What it does
node server.js	Run your Node.js file
nodemon server.js	Run with auto-restart on save
npm init -y	Create package.json
npm install express	Install Express
npm install mongoose	Install Mongoose
npm install cors	Allow cross-origin requests

<code>ng new my-app</code>	Create new Angular project
<code>ng serve</code>	Start Angular dev server (port 4200)
<code>ng generate component X</code>	Create new Angular component
<code>ng generate service X</code>	Create new Angular service
<code>app.get(path, handler)</code>	Express GET route
<code>app.post(path, handler)</code>	Express POST route
<code>mongoose.connect(uri)</code>	Connect to MongoDB
<code>Model.find()</code>	Get all documents
<code>Model.findById(id)</code>	Get one document
<code>new Model(data).save()</code>	Create a document
<code>Model.findByIdAndUpdate</code>	Update a document
<code>Model.findByIdAndDelete</code>	Delete a document

What to Learn Next

- JWT Authentication — secure your API with JSON Web Tokens
- Environment Variables — use dotenv to hide secrets
- Input Validation — use express-validator or Joi
- Angular Reactive Forms — build forms that validate in real-time
- Deployment — deploy backend to Render/Railway, frontend to Vercel/Netlify
- TypeScript on the backend — use ts-node for type-safe Node.js

■ You've got this!

The best way to learn MEAN is to build something. Start with a simple Todo app or a blog API. Break things, fix them, and you'll learn faster than any tutorial. Happy coding! ■